arXiv:2301.05217v3 [cs.LG] 19 Oct 2023

PROGRESS MEASURES FOR GROKKING VIA MECHANISTIC INTERPRETABILITY

Neel Nanda[*, †] **Lawrence Chan**[‡] **Tom Lieberum**[†] **Jess Smith**[†] **Jacob Steinhardt**[‡]

ABSTRACT

Neural networks often exhibit emergent behavior, where qualitatively new capabilities arise from scaling up the amount of parameters, training data, or training steps. One approach to understanding emergence is to find continuous *progress measures* that underlie the seemingly discontinuous qualitative changes. We argue that progress measures can be found via mechanistic interpretability: reverse-engineering learned behaviors into their individual components. As a case study, we investigate the recently-discovered phenomenon of "grokking" exhibited by small transformers trained on modular addition tasks. We fully reverse engineer the algorithm learned by these networks, which uses discrete Fourier transforms and trigonometric identities to convert addition to rotation about a circle. We confirm the algorithm by analyzing the activations and weights and by performing ablations in Fourier space. Based on this understanding, we define progress measures that allow us to study the dynamics of training and split training into three continuous phases: memorization, circuit formation, and cleanup. Our results show that grokking, rather than being a sudden shift, arises from the gradual amplification of structured mechanisms encoded in the weights, followed by the later removal of memorizing components.

1 INTRODUCTION

Neural networks often exhibit emergent behavior, in which qualitatively new capabilities arise from scaling up the model size, training data, or number of training steps (Steinhardt, 2022; Wei et al., 2022a). This has led to a number of breakthroughs, via capabilities such as in-context learning (Radford et al., 2019; Brown et al., 2020) and chain-of-thought prompting (Wei et al., 2022b). However, it also poses risks: Pan et al. (2022) show that scaling up the parameter count of models by as little as 30% can lead to emergent reward hacking.

Emergence is most surprising when it is abrupt, as in the case of reward hacking, chain-of-thought reasoning, or other phase transitions (Ganguli et al., 2022; Wei et al., 2022a). We could better understand and predict these phase transitions by finding *hidden progress measures* (Barak et al., 2022): metrics that precede and are causally linked to the phase transition, and which vary more smoothly. For example, Wei et al. (2022a) show that while large language models show abrupt jumps in their performance on many benchmarks, their cross-entropy loss decreases smoothly with model scale. However, cross-entropy does not explain *why* the phase changes happen.

In this work, we introduce a different approach to uncovering hidden progress measures: via *mechanistic explanations*.[1] A mechanistic explanation aims to reverse engineer the mechanisms of the network, generally by identifying the circuits (Cammarata et al., 2020; Elhage et al., 2021) within a model that implement a behavior. Using such explanations, we study *grokking*, where models abruptly transition to a generalizing solution after a large number of training steps, despite initially overfitting (Power et al., 2022). Specifically, we study modular addition, where a model takes inputs $a, b \in \{0, \ldots, P-1\}$ for some prime P and predicts their sum c mod P. Small transformers trained with weight decay on this task consistently exhibit grokking (Figure 2, Appendix C.2).

[*]Corresponding author, please direct correspondence to: neelnanda27@gmail.com

[†]Independent researcher.

[‡]University of California, Berkeley.

[1]Interactive versions of figures, as well as the code to reproduce our results, are available at https://neelnanda.io/grokking-paper.

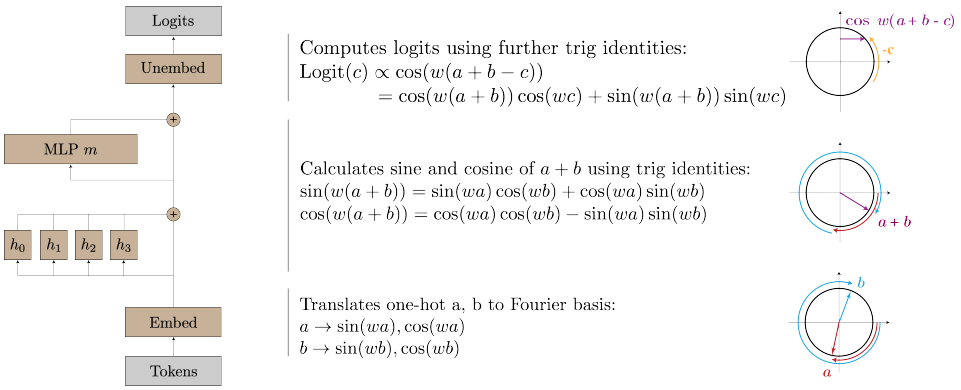

Figure 1: The algorithm implemented by the one-layer transformer for modular addition. Given two numbers a and b, the model projects each point onto a corresponding rotation using its embedding matrix. Using its attention and MLP layers, it then composes the rotations to get a representation of $a + b \mod P$. Finally, it "reads off" the logits for each $c \in \{0, 1, ..., P-1\}$, by rotating by $-c$ to get $\cos(w(a+b-c))$, which is maximized when $a + b \equiv c \mod P$ (since w is a multiple of $\frac{2\pi}{P}$).

We reverse engineer the weights of these transformers and find that they perform this task by mapping the inputs onto a circle and performing addition on the circle. Specifically, we show that the embedding matrix maps the inputs a, b to sines and cosines at a sparse set of *key frequencies* w_k. The attention and MLP layers then combine these using trigonometric identities to compute the sine and cosine of $w_k(a+b)$, and the output matrices shift and combine these frequencies.

We confirm this understanding with four lines of evidence (Section 4): (1) the network weights and activations exhibit a consistent periodic structure; (2) the neuron-logit map W_L is well approximated by a sum of sinusoidal functions of the key frequencies, and projecting the MLP activations onto these sinusoidal functions lets us "read off" trigonometric identities from the neurons; (3) the attention heads and MLP neuron are well approximated by degree-2 polynomials of trigonometric functions of a single frequency; and (4) ablating key frequencies used by the model reduces performance to chance, while ablating the other 95% of frequencies slightly *improves* performance.

Using our understanding of the learned algorithm, we construct two progress measures for the modular addition task—*restricted loss*, where we ablate every non-key frequency, and *excluded loss*, where we instead ablate all key frequencies. Both metrics improve continuously prior to when grokking occurs. We use these metrics to understand the training dynamics underlying grokking and find that training can be split into three phases: **memorization** of the training data; **circuit formation**, where the network learns a mechanism that generalizes; and **cleanup**, where weight decay removes the memorization components. Surprisingly, the sudden transition to perfect test accuracy in grokking occurs during cleanup, *after* the generalizing mechanism is learned. These results show that grokking, rather than being a sudden shift, arises from the gradual amplification of structured mechanisms encoded in the weights, followed by the later removal of memorizing components.

2 Related Work

Phase Changes. Recent papers have observed that neural networks quickly develop novel qualitative behaviors as they are scaled up or trained longer (Ganguli et al., 2022; Wei et al., 2022a). McGrath et al. (2021) find that AlphaZero quickly learns many human chess concepts between 10k and 30k training steps and reinvents human opening theory between 25k and 60k training steps.

Grokking. Grokking was first reported in Power et al. (2022), which trained two-layer transformers on several algorithmic tasks and found that test accuracy often increased sharply long after achieving perfect train accuracy. Millidge (2022) suggests that this may be due to SGD being a random walk on the optimal manifold. Our results echo Barak et al. (2022) in showing that the network instead makes continuous progress toward the generalizing algorithm. Liu et al. (2022) construct small examples of grokking, which they use to compute phase diagrams with four separate "phases" of learning. Thilak et al. (2022) argue that grokking can arise without explicit regularization, from an optimization anomaly they dub the slingshot mechanism, which may act as an implicit regularizer.

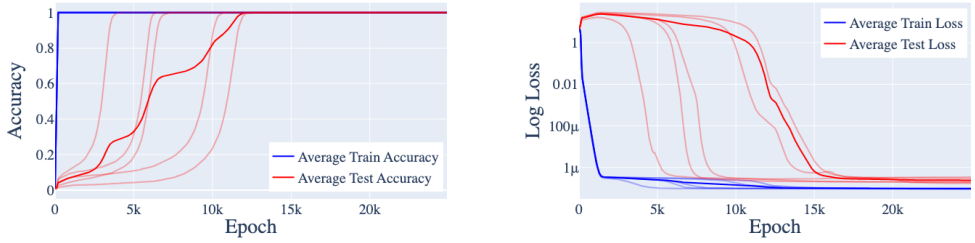

Figure 2: The train and test accuracy (left) and train and test loss (right) of one-layer transformers on the modular addition task described in Section 3, over 5 random seeds. These models consistently exhibit grokking: they quickly overfit early on in training, but then later learn to generalize.

Circuits-style mechanistic interpretability. The style of post-hoc mechanistic interpretability in Section 4 is heavily inspired by the Circuits approach of Cammarata et al. (2020), Elhage et al. (2021), and Olsson et al. (2022).

Progress measures. Barak et al. (2022) introduce the notion of *progress measures*—metrics that improve smoothly and that precede emergent behavior. They prove theoretically that training would amplify a certain mechanism and heuristically define a progress measure. In contrast, we use mechanistic intepretability to discover progress measures empirically.

3 Setup and Background

We train transformers to perform addition mod P. The input to the model is of the form "$a\ b\ =$", where a and b are encoded as P-dimensional one-hot vectors, and $=$ is a special token above which we read the output c. In our mainline experiment, we take $P = 113$ and use a one-layer ReLU transformer, token embeddings with $d = 128$, learned positional embeddings, 4 attention heads of dimension $d/4 = 32$, and $n = 512$ hidden units in the MLP. In other experiments, we vary the depth and dimension of the model. We did not use LayerNorm or tie our embed/unembed matrices.

Our mainline dataset consists of 30% of the entire set of possible inputs (that is, 30% of the $113 \cdot 113$ pairs of numbers mod P). We use full batch gradient descent using the AdamW optimizer (Loshchilov & Hutter, 2017) with learning rate $\gamma = 0.001$ and weight decay parameter $\lambda = 1$. We perform $40,000$ epochs of training. As there are only $113 \cdot 113$ possible pairs, we evaluate test loss and accuracy on all pairs of inputs not used for training.

Networks trained on this task consistently exhibit grokking. As Figure 2 shows, our networks first overfit the training set: train accuracy quickly converges to 100% and the train loss quickly declines, while the test accuracy remains low and the test loss remains high. After around $10,000$ epochs, the network generalizes and test accuracy increases to near 100%. In robustness experiments, we confirm that grokking consistently occurs for other architectures and prime moduli (Appendix C.2). In Section 5.3 we find that grokking does not occur without regularization.

To describe transformer components, we follow the conventions and notations laid out in Elhage et al. (2021). We focus on the $d \times p$ embedding matrix W_E, the $d \times n$ output matrix of the MLP layer W_{out}, and the $P \times d$ unembedding matrix W_U.[2] Let $\text{Logits}(a, b)$ denote the logit vector on inputs a, b, and $MLP(a, b)$ denote the MLP activations. Empirically, our networks do not significantly use the skip connection around the MLP (Appendix A.1), so $\text{Logits}(a, b) \approx W_U W_{out} \text{MLP}(a, b)$. We therefore also study the $P \times n$ neuron-logit map $W_L = W_U W_{out}$.

3.1 The Fourier multiplication algorithm

We claim that the learned networks use the following algorithm (Figure 1):

- Given two one-hot encoded tokens a, b map these to $\sin(w_k a)$, $\cos(w_k a)$, $\sin(w_k b)$, and $\cos(w_k b)$ using the embedding matrix, for various frequencies $w_k = \frac{2k\pi}{P}, k \in \mathbb{N}$.

[2] We ignore the embedding and unembedding of the '=' token for simplicity.